CCSDS 235.1-W-0.4 Interoperability Testing

Overview

This document defines the interoperability testing strategy for CCSDS 235.1-W-0.4 reference implementations. Interoperability ensures that different implementations can establish sessions, exchange data, and recover from errors correctly, validating compliance with the CCSDS specification.

Interoperability Goals

1. **Session Compatibility**: Implementations can establish sessions with each other
2. **Data Fidelity**: Data survives the encode-transmit-receive-decode cycle across implementations
3. **Specification Compliance**: All implementations produce identical outputs for identical inputs
4. **Cross-Platform Validation**: Implementations in different languages produce compatible results
5. **Error Recovery**: Implementations handle errors and recover correctly across implementations

Test Levels

Level 1: SPDU Interoperability

Objective: Verify all implementations encode/decode SPDUs identically

Test Procedure: 1. Each implementation encodes all SPDU types with known data 2. Export SPDUs in standardized format (binary, hexadecimal) 3. Each implementation decodes SPDUs from other implementations 4. Compare decoded data with original input

Pass Criteria: - Identical SPDU encoding across all implementations - Successful decoding of SPDUs from other implementations - All fields extracted correctly - Big-endian byte order consistent

Test SPDUs: - **Type F1 PLCW**: V(R)=42, expedited=3, PCID=1, retransmit=true - **Type F2 PLCW**: V(R)=1234, expedited=5, PCID=0, retransmit=false - **Type 1 SPDU**: SET TRANSMITTER PARAMETERS directive - **Type 2 SPDU**: TIME DISTRIBUTION directive - **Type 3 SPDU**: STATUS REPORT

Level 2: COP-P Interoperability

Objective: Verify FOP-P and FARM-P work correctly across implementations

Test Procedure: 1. Implementation A (FOP-P) sends sequence of frames 2. Implementation B (FARM-P) receives and validates sequences 3. Implementation B generates PLCWs 4. Implementation A processes PLCWs and retransmits if needed 5. Verify all frames delivered correctly

Pass Criteria: - Sequence numbers maintained correctly - Gaps detected and retransmit flag set - Retransmission recovers lost frames - All frames delivered in order (Sequence Controlled service) - Expedited frames bypass sequencing

Test Scenarios: - **Scenario 1**: No packet loss (baseline) - **Scenario 2**: Single packet loss (test retransmission) - **Scenario 3**: Multiple packet losses (test window management) - **Scenario 4**: Out-of-order delivery (test sequence validation) - **Scenario 5**: Resynchronization (test V(R) reset)

Level 3: Session Establishment Interoperability

Objective: Verify implementations can establish sessions with each other

Test Procedure: 1. Implementation A acts as Caller 2. Implementation B acts as Responder 3. Caller initiates hailing 4. Responder detects hailing and responds 5. Session established 6. Verify session state on both sides

Pass Criteria: - Hailing frames transmitted and detected correctly - Responder responds within timeout - Session transitions to Data Services phase - Both sides agree on session state - Callbacks invoked correctly

Test Configurations: - **Full Duplex**: Simultaneous bidirectional communication - **Half Duplex**: Alternating bidirectional communication - **Simplex**: Unidirectional communication

Level 4: Data Transfer Interoperability

Objective: Verify implementations can exchange data correctly

Test Procedure: 1. Establish session between implementations 2. Caller sends user data (U-frames) 3. Responder receives and validates data 4. Responder sends data back (if duplex mode allows) 5. Verify data integrity on both sides

Pass Criteria: - All data delivered correctly - No data corruption - Sequence Controlled service guarantees delivery - Expedited service bypasses sequencing - QoS flags honored

Test Data: - **Small data**: 100 bytes - **Medium data**: 10 KB - **Large data**: 1 MB - **Pattern data**: Alternating 0xAA/0x55 - **Random data**: Known seed for reproducibility

Level 5: Error Recovery Interoperability

Objective: Verify implementations handle errors and recover correctly

Test Procedure: 1. Establish session between implementations 2. Inject errors (packet loss, bit errors, timeouts) 3. Verify error detection 4. Verify error recovery (retransmission, reconnect, resynchronization) 5. Verify session continues or terminates gracefully

Pass Criteria: - Errors detected correctly - Retransmission recovers from packet loss - Reconnect recovers from communication loss - Resynchronization resets state correctly - Session terminates gracefully on fatal errors

Error Scenarios: - **Packet loss**: 1%, 5%, 10% loss rates - **Bit errors**: Random bit flips - **Timeout**: Hailing timeout, PLCW timeout - **Sequence errors**: Duplicate frames, out-of-order frames - **Invalid SPDUs**: Malformed SPDUs, invalid directives

Standardized Test Vector Format

SPDU Export Format

Filename: `spdu_{type}_{id}.bin`

Format: Binary file with metadata header

```
Header (64 bytes):
  - Magic: "CCSDS\0\0\0" (8 bytes)
  - Version: 1 (4 bytes)
  - SPDU Type: F1/F2/1/2/3/4/5 (4 bytes)
  - SPDU Length: N bytes (4 bytes)
  - Timestamp: Unix epoch (8 bytes)
  - Reserved: (36 bytes)

Data (N bytes):
  - SPDU bytes (big-endian)
```

JSON Metadata: `spdu_{type}_{id}.json`

```
{
  "version": "1.0",
  "timestamp": "2026-03-07T12:00:00Z",
  "spdu_type": "F1",
  "spdu_id": "test_001",
  "fields": {
```

```
    "report_value": 42,
    "expedited_counter": 3,
    "pcid": true,
    "retransmit_flag": true
  }
}
```

Frame Export Format

Filename: `frame_{type}_{id}.bin`

Format: Binary file with metadata header

```
Header (128 bytes):
  - Magic: "CCSDSFRM" (8 bytes)
  - Version: 1 (4 bytes)
  - Frame Type: P-frame/U-frame (4 bytes)
  - Frame Version: 3/4 (4 bytes)
  - Frame Length: N bytes (4 bytes)
  - QoS: Expedited/Sequence Controlled (4 bytes)
  - Sequence Number: (4 bytes)
  - Timestamp: Unix epoch (8 bytes)
  - Reserved: (80 bytes)

Data (N bytes):
  - Frame bytes
```

Session Export Format

Filename: `session_{id}.json`

Format: JSON

```
{
  "version": "1.0",
  "timestamp": "2026-03-07T12:00:00Z",
  "session_id": "test_session_001",
  "configuration": {
    "role": "caller",
    "duplex_mode": "full_duplex",
    "plcw_type": "F1",
    "hailing_timeout": 5000,
    "window_size": 10
  },
  "events": [
    {
      "time": 0.0,
      "event": "hailing_start"
    },
    {
      "time": 0.5,
      "event": "hailing_detected"
    },
    {
      "time": 1.0,
      "event": "session_established"
    },
```

```
    {
      "time": 2.0,
      "event": "data_sent",
      "data": "user_data_001"
    },
    {
      "time": 3.0,
      "event": "session_terminated"
    }
  ]
}
```


Interoperability Test Suite

Test Case 1: Basic SPDU Exchange

Purpose: Verify basic SPDU encoding/decoding

Inputs: - Type F1 PLCW with known values - Type F2 PLCW with known values - Type 1 SPDU with SET TRANSMITTER directive

Expected Outputs: - All implementations encode identically - All implementations decode correctly - All fields match original values

Test Case 2: COP-P Sequence Validation

Purpose: Verify FOP-P and FARM-P sequence handling

Inputs: - Sequence of 100 frames (V(S) = 0-99) - No packet loss

Expected Outputs: - All frames accepted by FARM-P - V(R) increments correctly (0 → 100) - No retransmit flag set - All frames delivered in order

Test Case 3: COP-P Retransmission

Purpose: Verify retransmission on packet loss

Inputs: - Sequence of 100 frames - Drop frame 50 (simulate packet loss)

Expected Outputs: - FARM-P detects gap (V(R) = 50, received = 51) - Retransmit flag set in PLCW - FOP-P retransmits frame 50 - All frames delivered correctly

Test Case 4: Session Establishment (Full Duplex)

Purpose: Verify session establishment in Full Duplex mode

Inputs: - Caller initiates hailing - Responder listens and responds

Expected Outputs: - Hailing frames transmitted persistently - Responder detects hailing within timeout - Responder responds with PLCW - Session established on both sides - State transitions to Data Services

Test Case 5: Session Establishment (Half Duplex)

Purpose: Verify session establishment in Half Duplex mode

Inputs: - Caller initiates hailing - Responder listens and responds

Expected Outputs: - Hailing frames transmitted persistently - Responder detects hailing within timeout - Responder responds when allowed to transmit - Session established on both sides - State transitions to Data Services

Test Case 6: Data Transfer (Expedited Service)

Purpose: Verify Expedited service bypasses sequencing

Inputs: - Send 10 frames with Expedited QoS - Frames sent out of order (intentionally)

Expected Outputs: - All frames delivered immediately - No sequence validation - Frames delivered in received order (not sent order) - Expedited counter increments (modulo-8)

Test Case 7: Data Transfer (Sequence Controlled Service)

Purpose: Verify Sequence Controlled service guarantees delivery

Inputs: - Send 100 frames with Sequence Controlled QoS - Simulate 5% packet loss

Expected Outputs: - All frames delivered correctly (after retransmission) - Frames delivered in order - Retransmission recovers lost frames - No data loss

Test Case 8: Reconnect

Purpose: Verify reconnect recovers from communication loss

Inputs: - Establish session - Transfer data - Simulate communication loss (no frames for timeout period) - Caller initiates reconnect

Expected Outputs: - Reconnect (rehailing) succeeds - Session state preserved (V(S), V(R) not reset) - Data transfer resumes - No data loss

Test Case 9: Resynchronization

Purpose: Verify resynchronization resets state

Inputs: - Establish session - Transfer data (V(R) = 50) - Caller sends resynchronization directive (new V(R) = 0)

Expected Outputs: - FARM-P resets V(R) to 0 - FOP-P resets V(S) to 0 - Data transfer continues with new sequence numbers - No sequence errors

Test Case 10: Multi-Channel Operations

Purpose: Verify PCID support for multiple physical channels

Inputs: - Configure two physical channels (PCID 0 and 1) - Send frames on both channels

Expected Outputs: - Frames routed to correct channels - PCID in PLCW identifies channel - Both channels operate independently - No cross-channel interference

Cross-Implementation Testing Protocol

Phase 1: Setup

1. Each team provides their implementation
2. Agree on test vector formats
3. Generate reference test vectors
4. Distribute test vectors to all teams
5. Set up test environment (TCP/UDP channels for ground testing)

Phase 2: Independent Testing

1. Each team runs test suite independently
2. Generate outputs in standardized formats

3. Document any deviations or issues
4. Submit results to coordination team

Phase 3: Cross-Validation

1. Exchange generated test vectors
2. Each implementation tests against others' outputs
3. Document compatibility matrix
4. Identify and resolve discrepancies

Phase 4: Live Testing

1. Set up TCP-based connections between implementations
2. Establish sessions between implementations
3. Transfer data and verify correctness
4. Test error recovery scenarios
5. Document results

Phase 5: Compliance Verification

1. Compare all outputs against CCSDS 235.1 specification
2. Validate state transitions
3. Measure performance metrics
4. Generate compliance report

Interoperability Metrics

SPDU Metrics

- **Encoding Match Rate**: Percentage of SPDUs matching reference (target: 100%)
- **Decoding Success Rate**: Percentage of SPDUs decoded correctly (target: 100%)

COP-P Metrics

- **Sequence Correctness**: Percentage of sequences validated correctly (target: 100%)
- **Retransmission Success**: Percentage of lost frames recovered (target: 100%)
- **Delivery Guarantee**: Percentage of frames delivered (Sequence Controlled) (target: 100%)

Session Metrics

- **Establishment Success**: Percentage of sessions established (target: $> 99\%$)
- **Hailing Time**: Time to establish session (target: < 5 seconds)
- **Reconnect Success**: Percentage of reconnects successful (target: $> 95\%$)

Data Transfer Metrics

- **Data Integrity**: Percentage of data delivered correctly (target: 100%)
- **Throughput**: Frames per second (target: > 100 fps)
- **Latency**: End-to-end latency (target: < 100ms)

Compliance Checklist

Specification Compliance

All SPDU types encode/decode per specification
PLCWs generated per specification format
FOP-P maintains V(S) and N(N) correctly

FARM-P maintains V(R) correctly
State transitions per CCSDS 235.1 tables
Hailing mechanism per specification
All duplex modes supported

Interoperability Compliance

SPDUs identical across implementations
Sessions established across implementations
Data exchanged successfully across implementations
Error recovery works across implementations
Test vectors compatible across implementations

Performance Compliance

SPDU encoding/decoding < 1ms
Frame processing < 1ms
Session overhead < 50KB
Throughput > 100 fps

Test Vector Repository Structure

```
test-vectors/
  spdus/
     type_f1/
        test_001.bin
        test_001.json
        ...
     type_f2/
     type_1/
     type_2/
     ...
  frames/
     p_frames/
        test_001.bin
        test_001.json
        ...
     u_frames/
  sessions/
     full_duplex/
        session_001.json
        ...
     half_duplex/
     simplex/
  reference/
      ccsds_examples/
      validation_data/
```

Reporting Format

Interoperability Test Report

Template:

```
# CCSDS 235.1-W-0.4 Interoperability Test Report
```

```
## Implementation Details
- Name: {Implementation Name}
- Version: {Version}
- Language: {Language}
- Date: {ISO-8601}

## Test Results Summary
- Total Tests: {N}
- Passed: {N}
- Failed: {N}
- Pass Rate: {X}%

## SPDU Interoperability Results
- SPDUs Tested: {N}
- Encoding Matches: {N}/{N}
- Decoding Success: {N}/{N}
- Match Rate: {X}%

## COP-P Interoperability Results
- Sequences Tested: {N}
- Sequence Correctness: {N}/{N}
- Retransmission Success: {N}/{N}
- Success Rate: {X}%

## Session Interoperability Results
- Sessions Tested: {N}
- Establishment Success: {N}/{N}
- Data Transfer Success: {N}/{N}
- Success Rate: {X}%

## Cross-Implementation Results
- Implementations Tested: {List}
- Compatibility Matrix:
  - Impl A  Impl B: {Pass/Fail}
  - Impl A  Impl C: {Pass/Fail}
  - Impl B  Impl C: {Pass/Fail}

## Issues Identified
1. {Issue description}
2. {Issue description}

## Compliance Status
- Specification Compliant: {Yes/No}
- Interoperable: {Yes/No}
- Performance Compliant: {Yes/No}
```

Continuous Integration

Automated Testing

1. Run test suite on every commit
2. Generate test vectors automatically
3. Compare against reference vectors
4. Report deviations immediately

Regression Testing

1. Maintain test vector archive
2. Test new versions against historical vectors
3. Ensure backward compatibility
4. Track performance over time

Ground-Based Testing Setup

TCP-Based Testing

Purpose: Multi-host testing without RF hardware

Setup: 1. Implementation A runs as TCP server (Responder) 2. Implementation B runs as TCP client (Caller) 3. Establish TCP connection 4. Run session establishment and data transfer 5. Verify results on both sides

Configuration:

```
{
  "physical_channel": {
    "type": "tcp",
    "host": "192.168.1.100",
    "port": 8235,
    "mode": "server"
  }
}
```

UDP-Based Testing

Purpose: Packet loss testing without RF hardware

Setup: 1. Implementation A sends frames via UDP 2. Implementation B receives frames via UDP 3. Simulate packet loss (drop random datagrams) 4. Verify COP-P retransmission recovers lost frames

Configuration:

```
{
  "physical_channel": {
    "type": "udp",
    "host": "192.168.1.100",
    "port": 8235,
    "packet_loss_rate": 0.05
  }
}
```

Conclusion

Interoperability testing ensures that CCSDS 235.1-W-0.4 implementations can work together seamlessly. By following this testing strategy and using standardized test vectors, teams can validate their implementations and ensure compliance with the specification.

The key to successful interoperability is: 1. **Standardized formats** for exchanging data 2. **Comprehensive test vectors** covering all scenarios 3. **Cross-validation** between implementations 4. **Live testing** with TCP/UDP channels 5. **Continuous testing** throughout development

Note: TCP and UDP physical channels are for ground-based testing only. Production space missions must use CCSDS 211.1 Proximity-1 Physical Layer.